

¿Qué es programar?

Pensamiento computacional

Los bloques fundamentales de todo programa: secuencia, decisión, repetición y variables — explicados sin código, para construir la base antes de escribir la primera línea de Python.

Nivel	Prerrequisitos	Versión	Fecha
Básico	Ninguno	v1.1	2026-08-28

01 Objetivos de aprendizaje

1. Explicar con sus propias palabras qué es un programa y qué es un algoritmo, usando una analogía cotidiana.
2. Identificar los tres bloques universales de todo programa: **secuencia**, **decisión** y **repetición**.
3. Distinguir un dato de una variable y explicar por qué una variable es como una caja etiquetada que puede cambiar de contenido.
4. Leer y describir en lenguaje natural un algoritmo sencillo escrito en pseudocódigo.
5. Construir, de forma conceptual, un algoritmo propio de 4-6 pasos aplicando al menos un condicional y un bucle.

02 ¿Qué es "programar", en realidad?

Programar es dar instrucciones extremadamente precisas a algo que no puede interpretar ambigüedades.

ANALOGÍA – LA RECETA DE COCINA

Le dejas una nota a un compañero que nunca ha cocinado para que haga un huevo frito. Si escribes "haz un huevo frito" sin detalle, fracasará: una computadora es exactamente ese compañero — ejecuta instrucciones al pie de la letra, sin sentido común.

Un **programa** es esa receta con un nivel de detalle altísimo. Un **algoritmo** es la receta en sí (los pasos), independiente de cualquier lenguaje. El **código** es esa misma receta traducida a un idioma que la máquina entiende — Python, a partir de la Semana 2.

03 Los tres bloques universales de todo programa

Todo programa —una app bancaria, un videojuego, un modelo de IA— se construye combinando solo tres bloques:

a) Secuencia

Los pasos ocurren uno tras otro, en orden. Como armar un mueble: base → patas → tapa.

b) Decisión (condicional)

El programa elige un camino u otro según una condición: "si llueve, llevo paraguas; si no, no lo llevo."

c) Repetición (bucle)

Se repite una acción hasta que se cumple (o deja de cumplirse) una condición: "mientras haya platos sucios, lavo un plato."

Progresión de ejemplos: servir un vaso de agua (solo secuencia) → cruzar la calle (secuencia + decisión) → revisar el correo hasta encontrar uno importante (secuencia + decisión + repetición).

EJEMPLO 4 – EL EXPLORADOR DEL LABERINTO

Imagina a un explorador dentro de un laberinto, linterna en mano, buscando la salida: *avanza un paso → si hay pared al frente, gira hacia el lado libre → si no hay pared, sigue de frente → repite hasta encontrar la salida.*

Lo memorable aquí es que el explorador **no sabe de antemano cuántos pasos dará** — puede ser 5 o 500. La repetición continúa exactamente hasta que se cumple la condición de salida, y en cada paso se vuelve a tomar una decisión, dentro de una secuencia estricta. Programar es, en el fondo, decirle a alguien exactamente cómo salir de un laberinto sin poder verlo tú mismo.

04 Datos y variables: cajas etiquetadas

ANALOGÍA – EL CASILLERO DE GIMNASIO

Un casillero con la etiqueta "EDAD" dice hoy "30" y mañana puede decir "31": el casillero (la variable) conserva su nombre, pero su contenido cambia.

Los **datos** son los valores (30, "Ana", verdadero/falso); las **variables** son las etiquetas que permiten reutilizarlos. En próximas semanas, `edad = 30` en código será la traducción sintáctica exacta de este casillero etiquetado.

05 Leer un algoritmo en pseudocódigo

El pseudocódigo es escribir algoritmos en español estructurado, sin la sintaxis exacta de un lenguaje real todavía.

PSEUDOCÓDIGO – ENTRADA A CINE PARA MAYORES DE EDAD

```
INICIO
  edad = preguntar la edad de la persona
  SI edad es mayor o igual a 18 ENTONCES
    mostrar "Puede entrar"
  SI NO
    mostrar "No puede entrar"
FIN
```

Veamos ahora un algoritmo intermedio, un poco más rico: un guardián revisando una fila de invitados en la entrada de un evento.

PSEUDOCÓDIGO – EL GUARDIÁN Y LA FILA VIP

```
INICIO
  fila_de_invitados = lista de personas esperando entrar
  MIENTRAS queden personas en la fila HACER
    invitado = siguiente persona de la fila
    SI el nombre de invitado está en la lista VIP ENTONCES
      mostrar "Bienvenido, " + invitado + ", pase directo"
    SI NO
      mostrar invitado + ", por favor espere en la fila general"
    quitar a invitado de la fila
  FIN
```

A diferencia del ejemplo del cine (una sola decisión y termina), aquí el guardián repite el mismo proceso **una vez por cada persona de la fila**, sin saber de antemano cuánta gente hay. Ya combina los tres bloques en un solo algoritmo.

Del pseudocódigo al código real: el guardián y la fila VIP

PYTHON

```
# Cada invitado es una tupla (nombre, es_vip): el dato "es VIP" viaja pegado a la persona.
fila_de_invitados = [
    ("Marta", True),
    ("Luis", False),
    ("Carla", False),
    ("Diego", True),
]

# REPETICIÓN: aquí está la diferencia clave con el paraguas. ALLÁ NO sabíamos de
# antemano cuántos días tocaría revisar, así que hacía falta un "while". Acá la
# fila YA existe completa: se sabe de entrada cuántos invitados hay, así que
# "recorrer la fila" se traduce de forma más natural con un "for".
for nombre, es_vip in fila_de_invitados:

    # DECISIÓN: "if ... else ..." traduce "SI está en la Lista VIP ENTONCES ... SI NO ..."
    if es_vip:
        print(f"Bienvenido, {nombre}, pase directo")
    else:
        print(f"{nombre}, por favor espere en la fila general")

# SECUENCIA: no hace falta escribir "quitar a invitado de la fila": el propio
# "for" ya avanza solo al siguiente invitado en cada vuelta.
```

DOS SABORES DE REPETICIÓN

while se usa cuando no se sabe de antemano cuántas vueltas hará falta dar — se repite *hasta que* pase algo (el paraguas). **for** se usa cuando se recorre una colección completa y conocida de antemano (la fila de invitados, o una lista de números). Es el mismo bloque lógico de "repetición"; Python solo ofrece dos sintaxis según el caso.

06 Por qué esto importa para IA/ML

Estos mismos tres bloques no desaparecen cuando el tema se vuelve más avanzado: son la base invisible de cómo funciona un sistema de inteligencia artificial. Más adelante en la academia verás que **entrenar un modelo es, en el fondo, un bucle que se repite miles o millones de veces**: el modelo revisa un ejemplo, compara su respuesta con la correcta, se ajusta un poco, y vuelve a empezar con el siguiente — igual que el explorador del laberinto repitiendo "avanzar y decidir" hasta encontrar la salida.

Cuando un modelo de IA "decide" algo (por ejemplo, si una foto muestra un gato o un perro), ejecuta una versión más sofisticada de nuestro bloque de decisión, solo que compara probabilidades en vez de una condición simple como "Si edad $\geq$ 18". Y todo ocurre en una secuencia ordenada de pasos, sin saltos ni magia. No hace falta entender aún los detalles técnicos: la lógica de hoy es literalmente el lenguaje con el que, meses más adelante, describirás cómo "piensa" una inteligencia artificial.

07 Ejemplo práctico: el paraguas

Problema: decidir si salir con paraguas, revisando el pronóstico día a día hasta encontrar lluvia.

PSEUDOCÓDIGO

```
INICIO
  día = 1
  encontrado_lluvia = falso
  MIENTRAS día sea menor o igual a 7 Y encontrado_lluvia sea falso HACER
    pronostico = consultar el pronóstico del día actual
    SI pronostico es "lluvia" ENTONCES
      mostrar "Lleva paraguas el día " + día
      encontrado_lluvia = verdadero
    SI NO
      día = día + 1
FIN
```

Del pseudocódigo al código real (Python)

PYTHON

```
# Lista simulada de pronósticos para 7 días (dato fijo por ahora).
pronosticos = ["soleado", "nublado", "soleado", "lluvia", "nublado", "soleado", "lluvia"]

# VARIABLES: mismos "casilleros etiquetados" del pseudocódigo.
día = 1
encontrado_lluvia = False

# REPETICIÓN: traducción directa de "MIENTRAS ... HACER"
while día <= 7 and not encontrado_lluvia:
    # SECUENCIA: ocurre siempre primero, en orden, en cada vuelta.
    pronostico = pronosticos[día - 1] # índice - 1: Python cuenta desde 0

    # DECISIÓN: traducción directa de "SI ... ENTONCES ... SI NO ..."
    if pronostico == "lluvia":
        print(f"Lleva paraguas el día {día}")
        encontrado_lluvia = True
    else:
        día = día + 1
```

PUNTO DE CONFUSIÓN CLÁSICO

día = día + 1 está dentro del **else** a propósito: solo se avanza de día si NO llovió. Distinguir = (asignación) de == (comparación) evita errores comunes al empezar.

Resultado al ejecutarlo: Lleva paraguas el día 4.

Segundo ejemplo: contar números pares

PYTHON

```
numeros = [3, 8, 15, 42, 7, 16]
contador_pares = 0

for numero in numeros:          # REPETICIÓN
    if numero % 2 == 0:         # DECISIÓN
        contador_pares = contador_pares + 1  # SECUENCIA

print(f"Hay {contador_pares} números pares en la lista")  # Hay 3 números pares en la lista
```

08 Notas de vigencia técnica (2026)

Contenido fundacional, sin partes desactualizadas: los tres bloques lógicos son atemporales y Python sigue siendo el lenguaje de entrada estándar a IA/ML. Para la Semana 2 se recomienda **Python 3.12+** y un entorno en línea tipo Jupyter/Google Colab o Replit, para no bloquear a nadie por instalación local.

09 Errores comunes de principiante

- **Creer que "programar" es lo mismo que "escribir código" desde el día uno.** Se puede razonar sobre algoritmos completos sin escribir una sola línea de sintaxis: la lógica viene primero.
- **Pensar que el orden de los pasos "no importa tanto".** Invertir dos pasos de una secuencia (cerrar la llave antes de llenar el vaso) puede arruinar el resultado, aunque la intención sea correcta.
- **Asumir que una variable tiene un solo valor para siempre.** Es habitual olvidar que una variable (como `día` o `encontrado_lluvia`) puede — y suele — cambiar varias veces durante la ejecución.
- **Confundir una decisión con una repetición.** Usar un "SI / SI NO" cuando el problema en realidad necesita repetir la acción varias veces (revisar a *todos* los invitados, no solo al primero) es uno de los errores lógicos más frecuentes al iniciar.

10 Ejercicio propuesto

PARA PRACTICAR POR TU CUENTA

Tienes esta lista con las edades de un grupo de personas en la fila de una montaña rusa:

```
edades = [12, 17, 8, 20, 15, 21]
```

Escribe un programa en Python que recorra la lista y, para cada persona, muestre si puede subir ($\text{edad} \geq 14$) o si debe esperar en la zona familiar. Al final, debe mostrar también cuántas personas del grupo sí pueden subir.

Pista: vas a necesitar los tres bloques de esta clase y una variable extra que funcione como "contador", igual que `contador_pares`.

11 Autoevaluación

1. ¿Cuál es la diferencia principal entre un algoritmo y el código?

El algoritmo es el plan/lógica independiente del lenguaje; el código es su traducción a sintaxis ejecutable.

2. Nombra los tres bloques universales y da un ejemplo cotidiano de cada uno (distinto a los de la clase).

Secuencia, decisión, repetición; cualquier ejemplo coherente con la definición es válido.

3. ¿Por qué una variable es un "casillero etiquetado" y no una caja de contenido fijo?

El nombre permanece igual, pero el valor almacenado puede cambiar durante la ejecución del programa.

4. Identifica el bloque de cada línea: a) tomar `temperatura_actual` b) Si `temperatura_actual > 30` ENTONCES mostrar "hace calor" c) MIENTRAS haya prendas en la canasta HACER doblar una prenda.

a) Secuencia, b) Decisión, c) Repetición.

5. Verdadero o falso: "Para aprender a programar es indispensable saber matemáticas avanzadas primero."

Falso: los fundamentos son lógicos y se explican con analogías cotidianas; las matemáticas de ML llegan mucho después en el roadmap.

6. Explica con tus propias palabras cómo los tres bloques se relacionan con el proceso de entrenar un modelo de inteligencia artificial, sin usar términos técnicos de programación.

El entrenamiento repite muchas veces el mismo ciclo (repetición): revisa un ejemplo, compara su respuesta con la correcta y se ajusta un poco antes de pasar al siguiente. En cada ejemplo también "decide" algo comparando probabilidades (decisión). Y todo ocurre paso a paso, en un orden fijo (secuencia), igual que en los algoritmos cotidianos de esta clase.

7. En el código del paraguas, ¿por qué `dia = dia + 1` está dentro del `else`?

Porque solo se debe avanzar de día si no llovió; si llovió, `encontrado_lluvia` ya detiene el bucle en la siguiente revisión.

8. En el ejemplo de números pares, ¿qué línea es secuencia, cuál decisión y cuál repetición?

```
for numero in numeros: repetición; if numero % 2 == 0: decisión; contador_pares =  
contador_pares + 1 secuencia.
```

9. En el ejemplo del guardián, el pseudocódigo usa MIENTRAS, pero el código Python usa `for`. ¿Es un error?

No. El pseudocódigo describe correctamente la idea general ("repetir mientras queden personas"), pero como la fila ya es una lista completa y conocida de antemano, Python permite expresar esa misma repetición de forma más directa con un `for`. Es la misma lógica, expresada con la herramienta más natural para el caso.

12 Verificación de dependencias

Válido como punto de partida: no requiere lenguaje de programación previo, no requiere matemáticas previas, no requiere herramientas instaladas, y es prerequisite lógico de todos los módulos siguientes del roadmap (Semana 2 en adelante).